

Verified multi-word compare-and-set and software transactional memory for OCAML 5

ANONYMOUS AUTHOR(S)

OCAML has recently acquired the ability to have multiple domains running in parallel. Libraries like EIO [Madhavapeddy and Leonard [n. d.]] and DOMAINSlib [Sivaramakrishnan [n. d.]] utilize OCAML's support for algebraic effects to provide lightweight threads of control. But, while threads are a prerequisite for concurrent programming, we also need mechanisms for threads to communicate and synchronize, such as message queues, mutexes and condition variables. However, such mechanisms do not compose and can be challenging to use. Transactional memory [Shavit and Touitou 1995] is a more recent abstraction that offers both a relatively familiar programming model and composability.

We present the KCAS library, a software transactional memory implementation for OCAML based on a state-of-the-art multi-word compare-and-set algorithm [Guerraoui, Kogan, Marathe and Zablotti 2020] enhanced with optimized read-only operations. KCAS features a convenient direct style interface and comes with a library of composable, reasonably well performing, concurrent data structures. It supports scheduler friendly blocking and timeouts.

We formally verify the core multi-word compare-and-set algorithm using the Iris [Jung, Krebbers, Jourdan, Bizjak, Birkedal and Dreyer 2018] concurrent separation logic. This effort involves advanced proof techniques, including logical atomicity [Jacobs and Piessens 2011] and prophecy variables [Jung, Lepigre, Parthasarathy, Rapoport, Timany, Dreyer and Jacobs 2020].

1 INTRODUCTION

1.1 Software transactional memory

Since its introduction in 1995 [Shavit and Touitou 1995], *software transactional memory* (STM) has been implemented in several programming languages, including Haskell [Harris, Marlow, Jones and Herlihy 2005], SCALA [Goes and the ZIO Contributors [n. d.]] and C++ [Granin [n. d.]].

```

type ('k, 'v) cache =
  { mutable space: int;
    table: ('k, 'k Dllist.node * 'v) Hashtbl.t;
    order: 'k Dllist.t;
  }

let get_opt { table; order; _ } key =
  Hashtbl.find_opt table key
  |> Option.map @@ fun (node, value) ->
    Dllist.move_1 node order ; value

type ('k, 'v) cache =
  { space: int Loc.t;
    table: ('k, 'k Dllist.Xt.node * 'v) Hashtbl.Xt.t;
    order: 'k Dllist.Xt.t;
  }

let get_opt ~xt { table; order; _ } key =
  Hashtbl.Xt.find_opt ~xt table key
  |> Option.map @@ fun (node, value) ->
    Dllist.Xt.move_1 ~xt node order ; value

```

1.2 Multi-word compare-and-set algorithm

Our implementation of STM, as described in more detail in [Section 3](#), relies on the *multi-word compare-and-set* (MCAS) algorithm, a generalization of the *single-word compare-and-set* (CAS) primitive: given a set of distinct shared-memory locations, each associated to an expected value and a desired value, this algorithm atomically either 1) updates all locations from expected to desired value and succeeds or 2) observes an unexpected value at some location and fails. However, while CAS is supported by most architectures, the multi-word variant has to be implemented at the software level.

It has been shown in the literature that MCAS can be made practical [[Harris, Fraser and Pratt 2002](#)]. Recent work [[Guerraoui, Kogan, Marathe and Zablotchi 2020](#)] further demonstrated that it can be implemented using only $k + 1$ (where k is the number of shared-memory locations) CAS in the uncontended case.

When building a transaction, loads, stores and more complex memory operations to be committed together atomically are translated to a MCAS operation. For instance, consider the two following transactions, involving locations a , b , x and y :

```

let x_to_b_sub_a ~xt () =
  let a = Xt.get ~xt a
  let b = Xt.get ~xt b in
  Xt.set ~xt x (b - a)

let y_to_a_add_b ~xt () =
  let a = Xt.get ~xt a
  let b = Xt.get ~xt b in
  Xt.set ~xt y (a + b)

```

Initially, a is set to 10, b to 52, x and y to 0. When they are committed, these transactions essentially correspond to the following MCAS operations:

```

CAS (a, 10, 10)      CAS (a, 10, 10)
CAS (b, 52, 52)      CAS (b, 52, 52)
CAS (x, 0, 42)        CAS (y, 0, 62)

```

CAS with equal expected and desired values essentially expresses an operation that does not change the logical content of the target location, but only "asserts" that it does not change during the operation.

One might then attempt to perform both MCAS operations in parallel. Unfortunately, this is not allowed by the MCAS implementations we are aware of. Indeed, every CAS actually updates the target locations. This means two things: 1) CAS operations targeting the same location can only execute sequentially; 2) CAS operations, even those that do not change the logical content of a location, cause contention as after the operation only the cache of the writer will have a valid copy of the location.

To address this issue, we extend upon the state-of-the-art algorithm [[Guerraoui, Kogan, Marathe and Zablotchi 2020](#)] to allow read-only CMP operations to be expressed directly and not write into memory. For instance, the two above transactions would generate the following operations, that can be run in parallel:

```

CMP (a, 10)           CMP (a, 10)
CMP (b, 52)           CMP (b, 52)
CAS (x, 0, 42)        CAS (y, 0, 62)

```

There is one drawback, however. This new algorithm is *obstruction-free* but not *lock-free* like the original one. In particular, two MCAS involving a common location may basically cancel each other indefinitely. To get the best of both worlds, we first attempt the MCAS operations in obstruction-free mode (CMP and CAS) and switch to lock-free mode (CAS only) after a number of failed attempts. The resulting algorithm therefore guarantees lock-free behavior.

1.3 Verified multi-word compare-and-set algorithm

1.4 Contributions

99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147

```

148 module Loc : sig
149   type 'a t
150   val make : ?padded:bool -> ?mode:Mode.t -> 'a -> 'a t
151   val get : 'a t -> 'a
152   val set : 'a t -> 'a -> unit
153   val compare_and_set : 'a t -> 'a -> 'a -> bool
154   val exchange : 'a t -> 'a -> 'a
155   val fetch_and_add : int t -> int -> int
156   val incr : int t -> unit
157   val decr : int t -> unit
158 end

```

Fig. 1. Interface for shared-memory locations (excerpt)

```

162 module Xt : sig
163   type 'x t
164   val get : xt:'x t -> 'a Loc.t -> 'a
165   val set : xt:'x t -> 'a Loc.t -> 'a -> unit
166   val update : xt:'x t -> 'a Loc.t -> ('a -> 'a) -> 'a
167   val modify : xt:'x t -> 'a Loc.t -> ('a -> 'a) -> unit
168   val exchange : xt:'x t -> 'a Loc.t -> 'a -> 'a
169   val swap : xt:'x t -> 'a Loc.t -> 'a Loc.t -> unit
170   val compare_and_set : xt:'x t -> 'a Loc.t -> 'a -> 'a -> bool
171   val compare_and_swap : xt:'x t -> 'a Loc.t -> 'a -> 'a -> 'a
172   val fetch_and_add : xt:'x t -> int Loc.t -> int -> int
173   val incr : xt:'x t -> int Loc.t -> unit
174   val decr : xt:'x t -> int Loc.t -> unit
175
176   val post_commit : xt:'x t -> (unit -> unit) -> unit
177   val validate : xt:'x t -> 'a Loc.t -> unit
178
179   type 'a tx = { tx : 'x. xt:'x t -> 'a } [@@unboxed]
180   val call : xt:'x t -> 'a tx -> 'a
181   val commit : ?timeoutf:float -> ?mode:Mode.t -> 'a tx -> 'a
182 end

```

Fig. 2. Interface for explicit transaction logs (excerpt)

2 OVERVIEW OF THE LIBRARY

2.1 Shared memory locations

2.2 Explicit transaction logs

transactional API

A transaction is a specification for generating a list of CAS

```

194 let transfer ~xt () =
195   let a' = Xt.get ~xt a

```

```
197     and b' = Xt.get ~xt b in
198     Xt.set ~xt s (a' + b')
199 Xt.commit { tx = transfer }
```

201 2.3 Standard concurrent data structures

202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245

3 IMPLEMENTATION OF THE LIBRARY

246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294

```

295 type 'a loc =
296   { atomic state: 'a state;
297     id: int;
298   }
299 and 'a state =
300   { casn: 'a casn;
301     mutable before: 'a;
302     mutable after: 'a;
303   }
304
305 and 'a cas =
306   { loc: 'a loc;
307     state: 'a state;
308   }
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343

```

Fig. 3. Type definitions for implementing multi-word compare-and-set

4 VERIFIED MULTI-WORD COMPARE-AND-SET ALGORITHM

4.1 Specification

4.2 Resource algebras

4.3 Proof sketch

4.4 Support for read-only operations

4.5 Physical equality taken seriously

```

344 1 let finish gid casn status =
345 2   match casn.status with
346 3   | Before -> false
347 4   | After -> true
348 5   | Undetermined _ as old_status ->
349 6     resolve (
350 7       Atomic.Loc.compare_and_set [%atomic.loc casn.status] old_status status
351 8     ) casn.proph (gid, status == After) |> ignore ;
352 9     casn.status == After
353 10
354 11 let rec determine_as casn cass =
355 12   let gid = ghost_id in
356 13   match cass with
357 14   | [] ->
358 15     finish gid casn After
359 16   | cas :: cass' ->
360 17     let { loc; state } = cas in
361 18     let state' = loc.state in
362 19     if state == state' then
363 20       determine_as casn cass'
364 21     else
365 22       let v = get_as state' in
366 23       if get_as state' != state.before then
367 24         finish gid casn Before
368 25       else
369 26         match casn.status with
370 27         | Before -> false
371 28         | After -> true
372 29         | Undetermined _ ->
373 30           if Atomic.Loc.compare_and_set [%atomic.loc loc.state] state' state
374 31           then determine_as casn cass'
375 32           else determine_as casn cass
376 33   and get_as state =
377 34     if determine state.casn then state.after else state.before
378 35   and determine casn =
379 36     match casn.status with
380 37     | Before -> false
381 38     | After -> true
382 39     | Undetermined cass -> determine_as casn cass
383
384
385
386
387
388
389
390
391
392

```

Fig. 4. Implementation of multi-word compare-and-set (1)


```

393 1 let make v id =
394 2   let _gid = ghost_id in
395 3   let casn = { status= After; proph= proph } in
396 4   let state = { casn; before= v; after= v } in
397 5   Atomic.make { state; id }
398 6
399 7 let get loc =
400 8   get_as loc.state
401 9
402 10 let cas cass =
403 11   let casn = { status= After; proph= proph } in
404 12   let cass =
405 13     Lst.map cass (fun (loc, before, after) ->
406 14       let state = { casn; before; after } in
407 15       { loc; state }
408 16     )
409 17   in
410 18   casn.status <- Undetermined cass ;
411 19   determine_aux casn cass
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441

```

Fig. 5. Implementation of multi-word compare-and-set (2)

5 BENCHMARKS

6 RELATED WORK

491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

7 CONCLUSION

540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588

REFERENCES

- John A. De Goes and the ZIO Contributors. [n. d.]. *ZIO*. <https://github.com/zio/zio>
- Alexander Granin. [n. d.]. *cpp_stm_free*. https://github.com/graninas/cpp_stm_free
- Rachid Guerraoui, Alex Kogan, Virendra J. Marathe, and Igor Zablotchi. 2020. Efficient Multi-Word Compare and Swap. In *34th International Symposium on Distributed Computing, DISC 2020, October 12–16, 2020, Virtual Conference (LIPIcs, Vol. 179)*, Hagit Attiya (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 4:1–4:19. <https://doi.org/10.4230/LIPICS.DISC.2020.4>
- Tim Harris, Simon Marlow, Simon L. Peyton Jones, and Maurice Herlihy. 2005. Composable memory transactions. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2005, June 15–17, 2005, Chicago, IL, USA*, Keshav Pingali, Katherine A. Yelick, and Andrew S. Grimshaw (Eds.). ACM, 48–60. <https://doi.org/10.1145/1065944.1065952>
- Timothy L. Harris, Keir Fraser, and Ian A. Pratt. 2002. A Practical Multi-word Compare-and-Swap Operation. In *Distributed Computing, 16th International Conference, DISC 2002, Toulouse, France, October 28–30, 2002 Proceedings (Lecture Notes in Computer Science, Vol. 2508)*, Dahlia Malkhi (Ed.). Springer, 265–279. https://doi.org/10.1007/3-540-36108-1_18
- Bart Jacobs and Frank Piessens. 2011. Expressive modular fine-grained concurrency specification. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2011, Austin, TX, USA, January 26–28, 2011*, Thomas Ball and Mooly Sagiv (Eds.). ACM, 271–282. <https://doi.org/10.1145/1926385.1926417>
- Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. <https://doi.org/10.1017/S0956796818000151>
- Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. 2020. The future is ours: prophecy variables in separation logic. *Proc. ACM Program. Lang.* 4, POPL (2020), 45:1–45:32. <https://doi.org/10.1145/3371113>
- Anil Madhavapeddy and Thomas Leonard. [n. d.]. *Eio*. <https://github.com/ocaml-multicore/eio>
- Nir Shavit and Dan Touitou. 1995. Software Transactional Memory. In *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing, Ottawa, Ontario, Canada, August 20–23, 1995*, James H. Anderson (Ed.). ACM, 204–213. <https://doi.org/10.1145/224964.224987>
- K. C. Sivaramakrishnan. [n. d.]. *Domainslib*. <https://github.com/ocaml-multicore/domainslib>